

🦊 ДОМЕН 11: GO (GOLANG) — ПОЛНАЯ ДЕТАЛИЗАЦИЯ (ОТ НУЛЯ ДО ЭКСПЕРТА)

СТРУКТУРА КАЖДОЙ ТЕМЫ

Для каждой темы я даю:

- **Что нужно знать** — теория для понимания
 - **Вопросы на собеседовании** (Junior → Senior → Staff)
 - **Практика** — конкретные задания для закрепления
-

РАЗДЕЛ 11.1: ОСНОВЫ ЯЗЫКА (JUNIOR LEVEL)

11.1.1. Синтаксис и базовые конструкции (Junior Level)

№ 11.1.1.1 — Переменные

- **Что нужно знать:** var (явное объявление), := (короткое объявление), типы (int, float64, string, bool, byte, rune)
- **Вопросы на собеседовании:** "В чем разница между var и :=?"
- **Практика:** Написать программу, объявляющую переменные разных типов

№ 11.1.1.2 — Константы

- **Что нужно знать:** const, iota (генератор перечислений)
- **Вопросы на собеседовании:** "Что делает iota?"
- **Практика:** Создать перечисление с iota для дней недели

№ 11.1.1.3 — Функции

- **Что нужно знать:** объявление, параметры, возврат нескольких значений, именованные возвращаемые значения
- **Вопросы на собеседовании:** "Как вернуть несколько значений из функции?"
- **Практика:** Написать функцию, возвращающую (int, error)

№ 11.1.1.4 — Управляющие конструкции

- **Что нужно знать:** if, for (единственный цикл в Go), switch
- **Вопросы на собеседовании:** "Как написать цикл for в Go?"
- **Практика:** Написать программу для вычисления факториала

№ 11.1.1.5 — Пакеты

- **Что нужно знать:** `package main`, `import`, экспортируемые идентификаторы (с большой буквы)
- **Вопросы на собеседовании:** "Какие имена экспортируются из пакета?"
- **Практика:** Создать свой пакет `math` и импортировать его

№ 11.1.1.6 — Типы данных

- **Что нужно знать:** `struct` (структуры), методы, встраивание (`embedding`)
- **Вопросы на собеседовании:** "Как добавить метод к структуре?"
- **Практика:** Создать структуру `User` с полями `Name`, `Age` и методом `Greet()`

№ 11.1.1.7 — Указатели

- **Что нужно знать:** `*` (разыменование), `&` (взятие адреса), `new()`
- **Вопросы на собеседовании:** "В чем разница между указателем и значением?"
- **Практика:** Написать функцию, изменяющую значение по указателю

№ 11.1.1.8 — Массивы и срезы

- **Что нужно знать:** `[n]type` (массив фиксированной длины), `[]type` (срез), `make()`, `append()`, `copy()`
- **Вопросы на собеседовании:** "В чем разница между массивом и срезом?"
- **Практика:** Написать программу, работающую со срезами (`append`, `copy`, `slice`)

№ 11.1.1.9 — Map (хеш-таблица)

- **Что нужно знать:** `map[keyType]valueType`, `make()`, добавление, удаление (`delete`)
- **Вопросы на собеседовании:** "Как удалить элемент из `map`?"
- **Практика:** Создать `map map[string]int` для подсчета слов

№ 11.1.1.10 — Обработка ошибок

- **Что нужно знать:** `error` (интерфейс), `fmt.Errorf`, `errors.New`, паника и восстановление (`panic/recover`)
- **Вопросы на собеседовании:** "Как обрабатывать ошибки в Go?"

- **Практика:** Написать функцию с обработкой ошибок
-

11.1.2. Интерфейсы и композиция (Junior → Middle Level)

№ 11.1.2.1 — Интерфейсы

- **Что нужно знать:** определение, неявная реализация (duck typing), пустой интерфейс (`interface{}` → `any` в Go 1.18+)
- **Вопросы на собеседовании:** "Как реализуется интерфейс в Go?"
- **Практика:** Создать интерфейс `Reader` и реализовать его в структуре

№ 11.1.2.2 — Встраивание (Embedding)

- **Что нужно знать:** композиция вместо наследования
- **Вопросы на собеседовании:** "В чем отличие встраивания от наследования?"
- **Практика:** Встроить структуру `Person` в структуру `Employee`

№ 11.1.2.3 — Type assertions

- **Что нужно знать:** `value, ok := interface.(Type)`
- **Вопросы на собеседовании:** "Как проверить, реализует ли объект интерфейс?"
- **Практика:** Проверить, что переменная реализует `io.Writer`

№ 11.1.2.4 — Type switches

- **Что нужно знать:** `switch v := x.(type)`
 - **Вопросы на собеседовании:** "Как обработать несколько типов в switch?"
 - **Практика:** Написать функцию, обрабатывающую `int`, `string`, `float64`
-

11.1.3. Практические задания (Junior Level)

№ 11.1.3.1

- **Задание:** Написать программу "Hello World" с пакетом `main` и функцией `main()`
- **Что проверяет:** Знание структуры Go-программы

№ 11.1.3.2

- **Задание:** Создать структуру Rectangle с полями Width, Height и методом Area()
- **Что проверяет:** Знание struct и методов

№ 11.1.3.3

- **Задание:** Написать функцию, принимающую срез []int и возвращающую сумму
- **Что проверяет:** Работа со срезами

№ 11.1.3.4

- **Задание:** Написать программу, читающую файл и выводящую его содержимое
- **Что проверяет:** Работа с файлами (ioutil/os)

РАЗДЕЛ 11.2: ПАРАЛЛЕЛИЗМ (CONCURRENCY) (MIDDLE → SENIOR)

11.2.1. Goroutines и Channels (Middle Level)

№ 11.2.1.1 — Goroutines

- **Что нужно знать:** легковесные потоки, запуск с go, планировщик (GOMAXPROCS)
- **Вопросы на собеседовании:** "Что такое goroutine?"
- **Практика:** Написать программу с 10 goroutine, выводящими свои ID

№ 11.2.1.2 — Channels

- **Что нужно знать:** создание (make(chan T)), буферизированные vs небуферизированные, отправка (<-), получение (<-)
- **Вопросы на собеседовании:** "В чем разница между буферизированным и небуферизированным каналом?"
- **Практика:** Написать программу, передающую данные через канал

№ 11.2.1.3 — select

- **Что нужно знать:** мультиплексирование каналов, default (неблокирующая операция)
- **Вопросы на собеседовании:** "Как работать с несколькими каналами?"
- **Практика:** Написать программу, обрабатывающую два канала с select

№ 11.2.1.4 — close

- **Что нужно знать:** закрытие канала, чтение из закрытого канала
- **Вопросы на собеседовании:** "Что происходит при чтении из закрытого канала?"
- **Практика:** Написать программу с закрытием канала и обработкой ok

№ 11.2.1.5 — sync.WaitGroup

- **Что нужно знать:** ожидание завершения группы goroutine
- **Вопросы на собеседовании:** "Как дождаться завершения всех goroutine?"
- **Практика:** Написать программу с WaitGroup для 10 goroutine

№ 11.2.1.6 — sync.Mutex

- **Что нужно знать:** мьютексы для защиты общих данных
- **Вопросы на собеседовании:** "Как защитить общие данные от race condition?"
- **Практика:** Написать программу с инкрементом счетчика через Mutex

№ 11.2.1.7 — sync.RWMutex

- **Что нужно знать:** read-write lock
- **Вопросы на собеседовании:** "Когда использовать RWMutex?"
- **Практика:** Реализовать кеш с RWMutex

№ 11.2.1.8 — context

- **Что нужно знать:** управление временем жизни, отмена операций, таймауты
- **Вопросы на собеседовании:** "Как отменить выполнение goroutine?"
- **Практика:** Написать программу с context.WithCancel и context.WithTimeout

11.2.2. Продвинутый параллелизм (Senior Level)

№ 11.2.2.1 — Worker Pool

- **Что нужно знать:** паттерн для ограничения количества goroutine

- **Вопросы на собеседовании:** "Как ограничить количество параллельных задач?"
- **Практика:** Реализовать worker pool с каналом задач

№ 11.2.2.2 — Fan-in / Fan-out

- **Что нужно знать:** объединение/разделение потоков данных
- **Вопросы на собеседовании:** "Что такое fan-in и fan-out?"
- **Практика:** Реализовать объединение результатов из нескольких каналов

№ 11.2.2.3 — Pipeline

- **Что нужно знать:** цепочка обработки данных через каналы
- **Вопросы на собеседовании:** "Как организовать pipeline в Go?"
- **Практика:** Реализовать pipeline: чтение → обработка → запись

№ 11.2.2.4 — sync.Once

- **Что нужно знать:** однократное выполнение функции
- **Вопросы на собеседовании:** "Как выполнить функцию только один раз?"
- **Практика:** Реализовать синглтон с sync.Once

№ 11.2.2.5 — sync.Pool

- **Что нужно знать:** пул объектов для переиспользования (уменьшение аллокаций)
- **Вопросы на собеседовании:** "Зачем нужен sync.Pool?"
- **Практика:** Написать программу с sync.Pool для буферов

№ 11.2.2.6 — atomic

- **Что нужно знать:** атомарные операции (AddInt32, CompareAndSwap, Load, Store)
- **Вопросы на собеседовании:** "Как инкрементировать счетчик без мьютекса?"
- **Практика:** Использовать atomic.AddInt64 для счетчика

№ 11.2.2.7 — errgroup

- **Что нужно знать:** обработка ошибок в группе goroutine

- **Вопросы на собеседовании:** "Как обрабатывать ошибки в параллельных задачах?"
 - **Практика:** Использовать golang.org/x/sync/errgroup
-

11.2.3. Практические задания (Middle → Senior Level)

№ 11.2.3.1

- **Задание:** Написать программу, загружающую 1000 URL параллельно с ограничением (10 параллельных)
- **Что проверяет:** Знание goroutine и worker pool

№ 11.2.3.2

- **Задание:** Реализовать producer-consumer с каналами
- **Что проверяет:** Знание каналов

№ 11.2.3.3

- **Задание:** Написать программу с таймаутом на выполнение через context
- **Что проверяет:** Знание context

№ 11.2.3.4

- **Задание:** Реализовать pipeline для обработки чисел (умножение → суммирование)
 - **Что проверяет:** Знание pipeline
-

РАЗДЕЛ 11.3: ВЕБ-РАЗРАБОТКА (MIDDLE → SENIOR)

11.3.1. HTTP и REST API (Middle Level)

№ 11.3.1.1 — net/http

- **Что нужно знать:** создание сервера, `http.HandleFunc`, `http.ListenAndServe`
- **Вопросы на собеседовании:** "Как создать HTTP-сервер в Go?"
- **Практика:** Написать HTTP-сервер с эндпоинтом `/hello`

№ 11.3.1.2 — Роутеры

- **Что нужно знать:** `http.ServeMux`, `chi` (популярный роутер), `gin` (веб-фреймворк)
- **Вопросы на собеседовании:** "Какие роутеры есть в Go?"
- **Практика:** Создать API с `chi` и REST-методами

№ 11.3.1.3 — Middleware

- **Что нужно знать:** логирование, аутентификация, CORS, восстановление после паники
- **Вопросы на собеседовании:** "Как написать middleware в Go?"
- **Практика:** Написать middleware для логирования запросов

№ 11.3.1.4 — JSON

- **Что нужно знать:** `encoding/json` (`Marshal/Unmarshal`), теги (`json:"name"`), `omitempty`
- **Вопросы на собеседовании:** "Как сериализовать struct в JSON?"
- **Практика:** Создать API, возвращающее JSON

№ 11.3.1.5 — Шаблоны

- **Что нужно знать:** `html/template`, безопасное рендеринг, `text/template`
- **Вопросы на собеседовании:** "Как рендерить HTML в Go?"
- **Практика:** Создать страницу с шаблоном

№ 11.3.1.6 — Статические файлы

- **Что нужно знать:** `http.FileServer`
- **Вопросы на собеседовании:** "Как отдавать статические файлы?"
- **Практика:** Отдавать статику из папки `static/`

№ 11.3.1.7 — Тестирование

- **Что нужно знать:** `net/http/httptest` для тестирования HTTP-серверов
- **Вопросы на собеседовании:** "Как тестировать HTTP-сервер?"
- **Практика:** Написать тест для эндпоинта `/hello`

11.3.2. gRPC и микросервисы (Senior Level)

№ 11.3.2.1 — gRPC

- **Что нужно знать:** `Protocol Buffers`, `protoc` (генерация кода)

- **Вопросы на собеседовании:** "Как создать gRPC-сервис на Go?"
- **Практика:** Создать gRPC-сервис UserService

№ 11.3.2.2 — Protocol Buffers

- **Что нужно знать:** .proto файлы, типы (string, int32, repeated), optional
- **Вопросы на собеседовании:** "Как определить gRPC-метод?"
- **Практика:** Написать .proto для сервиса

№ 11.3.2.3 — gRPC (4 типа)

- **Что нужно знать:** Unary, Server Streaming, Client Streaming, Bidirectional Streaming
- **Вопросы на собеседовании:** "Когда использовать Server Streaming?"
- **Практика:** Реализовать server streaming для больших данных

№ 11.3.2.4 — grpc-gateway

- **Что нужно знать:** REST + gRPC в одном сервисе
- **Вопросы на собеседовании:** "Как совместить REST и gRPC?"
- **Практика:** Настроить grpc-gateway для REST-доступа

№ 11.3.2.5 — Service Discovery

- **Что нужно знать:** consul, etcd для регистрации сервисов
- **Вопросы на собеседовании:** "Как реализовать Service Discovery?"
- **Практика:** Зарегистрировать сервис в Consul

№ 11.3.2.6 — Кеширование

- **Что нужно знать:** Redis (go-redis), кеширование ответов
- **Вопросы на собеседовании:** "Как кешировать ответы API?"
- **Практика:** Интегрировать Redis для кеширования

11.3.3. Практические задания (Middle → Senior Level)

№ 11.3.3.1

- **Задание:** Создать REST API на Go с роутингом, middleware и JSON-ответами
- **Что проверяет:** Знание net/http

№ 11.3.3.2

- **Задание:** Создать gRPC-сервис и клиент на Go
- **Что проверяет:** Знание gRPC

№ 11.3.3.3

- **Задание:** Написать middleware для аутентификации (JWT)
- **Что проверяет:** Знание безопасности

№ 11.3.3.4

- **Задание:** Интегрировать Redis для кеширования ответов
- **Что проверяет:** Знание Redis

РАЗДЕЛ 11.4: РАБОТА С ДАННЫМИ (MIDDLE → SENIOR)

11.4.1. SQL и ORM (Middle Level)

№ 11.4.1.1 — database/sql

- **Что нужно знать:** работа с БД, подключение, Query, QueryRow, Exec, Prepare
- **Вопросы на собеседовании:** "Как работать с SQL в Go?"
- **Практика:** Написать CRUD для PostgreSQL

№ 11.4.1.2 — SQL драйверы

- **Что нужно знать:** pq (PostgreSQL), go-sqlite3, mysql
- **Вопросы на собеседовании:** "Какие драйверы для SQL существуют?"
- **Практика:** Подключиться к PostgreSQL с pq

№ 11.4.1.3 — GORM

- **Что нужно знать:** ORM для Go, миграции, связи (has-one, has-many, belongs-to)
- **Вопросы на собеседовании:** "Что такое GORM?"
- **Практика:** Создать модель User с GORM и выполнить запросы

№ 11.4.1.4 — sqlx

- **Что нужно знать:** расширение для database/sql (удобные методы)
- **Вопросы на собеседовании:** "В чем преимущество sqlx?"

- **Практика:** Переписать код с database/sql на sqlx

№ 11.4.1.5 — Миграции

- **Что нужно знать:** migrate (golang-migrate), управление схемой БД
 - **Вопросы на собеседовании:** "Как управлять миграциями в Go?"
 - **Практика:** Настроить миграции с golang-migrate
-

11.4.2. NoSQL и очереди (Senior Level)

№ 11.4.2.1 — MongoDB

- **Что нужно знать:** mongo-go-driver, работа с коллекциями
- **Вопросы на собеседовании:** "Как работать с MongoDB в Go?"
- **Практика:** Написать CRUD для MongoDB

№ 11.4.2.2 — Redis

- **Что нужно знать:** go-redis, работа с типами (String, Hash, List, Set, Sorted Set)
- **Вопросы на собеседовании:** "Как работать с Redis в Go?"
- **Практика:** Использовать Redis для кеширования

№ 11.4.2.3 — RabbitMQ

- **Что нужно знать:** amqp, producer/consumer
- **Вопросы на собеседовании:** "Как работать с RabbitMQ в Go?"
- **Практика:** Написать producer/consumer для RabbitMQ

№ 11.4.2.4 — Kafka

- **Что нужно знать:** sarama (клиент), producer/consumer, consumer groups
 - **Вопросы на собеседовании:** "Как работать с Kafka в Go?"
 - **Практика:** Написать producer и consumer для Kafka
-

11.4.3. Практические задания (Middle → Senior Level)

№ 11.4.3.1

- **Задание:** Написать CRUD для PostgreSQL с GORM
- **Что проверяет:** Знание GORM

№ 11.4.3.2

- **Задание:** Написать producer/consumer для RabbitMQ
- **Что проверяет:** Знание очередей

№ 11.4.3.3

- **Задание:** Интегрировать Redis для кеширования запросов к БД
 - **Что проверяет:** Знание кеширования
-

РАЗДЕЛ 11.5: ИНСТРУМЕНТЫ И ЭКОСИСТЕМА (MIDDLE → SENIOR)

11.5.1. Инструменты (Middle Level)

№ 11.5.1.1 — go mod

- **Что нужно знать:** управление зависимостями, go.mod, go.sum, go get, go tidy
- **Вопросы на собеседовании:** "Как управлять зависимостями в Go?"
- **Практика:** Создать модуль и добавить зависимость

№ 11.5.1.2 — go build

- **Что нужно знать:** сборка приложений, кросс-компиляция (GOOS=linux GOARCH=amd64)
- **Вопросы на собеседовании:** "Как собрать Go-приложение для Linux?"
- **Практика:** Собрать бинарник для Linux на Windows

№ 11.5.1.3 — go test

- **Что нужно знать:** написание тестов, testing.T, табличные тесты, go test -cover
- **Вопросы на собеседовании:** "Как писать тесты в Go?"
- **Практика:** Написать тесты для своей функции

№ 11.5.1.4 — go fmt / go vet

- **Что нужно знать:** форматирование и проверка кода
- **Вопросы на собеседовании:** "Как форматировать код в Go?"
- **Практика:** Запустить go fmt на своем проекте

№ 11.5.1.5 — go run

- **Что нужно знать:** быстрый запуск
- **Вопросы на собеседовании:** "Как запустить Go-программу?"
- **Практика:** Запустить свою программу через `go run`

№ 11.5.1.6 — `go generate`

- **Что нужно знать:** генерация кода
- **Вопросы на собеседовании:** "Что делает `go generate`?"
- **Практика:** Использовать `go generate` для генерации кода из `.proto`

№ 11.5.1.7 — `go mod vendor`

- **Что нужно знать:** вендоринг зависимостей
- **Вопросы на собеседовании:** "Что такое вендоринг?"
- **Практика:** Скопировать зависимости в папку `vendor/`

№ 11.5.1.8 — Profiling

- **Что нужно знать:** `pprof` (профилирование CPU, памяти, блокировок)
- **Вопросы на собеседовании:** "Как профилировать Go-приложение?"
- **Практика:** Добавить `pprof` в приложение и снять профиль CPU

11.5.2. DevOps и инфраструктура (Senior Level)

№ 11.5.2.1 — Docker

- **Что нужно знать:** контейнеризация Go-приложений (`scratch image` для минимального размера)
- **Вопросы на собеседовании:** "Как контейнеризовать Go-приложение?"
- **Практика:** Создать `Dockerfile` с `multi-stage` для Go

№ 11.5.2.2 — Kubernetes

- **Что нужно знать:** деплой Go-приложения в K8s
- **Вопросы на собеседовании:** "Как деплоить Go-приложение в K8s?"
- **Практика:** Написать `Deployment` и `Service` для Go-приложения

№ 11.5.2.3 — CI/CD

- **Что нужно знать:** GitLab CI / GitHub Actions для Go
- **Вопросы на собеседовании:** "Как настроить CI для Go?"

- **Практика:** Написать пайплайн для тестов и сборки

№ 11.5.2.4 — Prometheus

- **Что нужно знать:** метрики для Go-приложений
- **Вопросы на собеседовании:** "Как добавить метрики в Go?"
- **Практика:** Добавить Prometheus metrics в приложение

№ 11.5.2.5 — OpenTelemetry

- **Что нужно знать:** трейсинг для Go
- **Вопросы на собеседовании:** "Как добавить трейсинг в Go?"
- **Практика:** Добавить OpenTelemetry для трассировки запросов

11.5.3. Практические задания (Middle → Senior Level)

№ 11.5.3.1

- **Задание:** Контейнеризовать Go-приложение с multi-stage сборкой
- **Что проверяет:** Знание Docker

№ 11.5.3.2

- **Задание:** Настроить CI/CD для Go-приложения (тесты, сборка, деплой)
- **Что проверяет:** Знание CI/CD

№ 11.5.3.3

- **Задание:** Добавить Prometheus метрики в Go-приложение
- **Что проверяет:** Знание мониторинга

№ 11.5.3.4

- **Задание:** Добавить профилирование pprof и снять дампы
- **Что проверяет:** Знание профилирования

РАЗДЕЛ 11.6: ЭКСПЕРТНЫЙ УРОВЕНЬ (STAFF LEVEL)

11.6.1. Внутреннее устройство Go (Staff Level)

№ 11.6.1.1 — Планировщик (Scheduler)

- **Что нужно знать:** GMP (Goroutine, Machine, Processor), work stealing, GOMAXPROCS

- **Вопросы на собеседовании:** "Как работает планировщик Go?"
- **Практика:** Написать программу, демонстрирующую работу планировщика

№ 11.6.1.2 — GC (Garbage Collector)

- **Что нужно знать:** триколорная маркировка, STW (Stop The World), настройка (GOGC)
- **Вопросы на собеседовании:** "Как работает GC в Go?"
- **Практика:** Включить логи GC и проанализировать

№ 11.6.1.3 — chan внутренности

- **Что нужно знать:** hchan структура, waitq, буферизация
- **Вопросы на собеседовании:** "Как устроен канал внутри?"
- **Практика:** Проанализировать структуру канала в исходниках Go

№ 11.6.1.4 — defer

- **Что нужно знать:** как работает, стек defer, паника и восстановление
- **Вопросы на собеседовании:** "Как работает defer?"
- **Практика:** Написать программу с несколькими defer и показать порядок

№ 11.6.1.5 — interface внутренности

- **Что нужно знать:** eface (пустой интерфейс), iface (интерфейс с методами), таблица методов
- **Вопросы на собеседовании:** "Как устроен интерфейс внутри?"
- **Практика:** Проанализировать структуру интерфейса в исходниках

№ 11.6.1.6 — slice внутренности

- **Что нужно знать:** структура слайса (ptr, len, cap), рост слайса
- **Вопросы на собеседовании:** "Как устроен слайс внутри?"
- **Практика:** Проанализировать рост слайса при append

№ 11.6.1.7 — map внутренности

- **Что нужно знать:** хеш-таблица, bucket, overflow
- **Вопросы на собеседовании:** "Как устроен map внутри?"
- **Практика:** Проанализировать структуру map в исходниках

№ 11.6.1.8 — sync.Mutex

- **Что нужно знать:** внутреннее устройство, fair/unfair
 - **Вопросы на собеседовании:** "Как устроен Mutex внутри?"
 - **Практика:** Проанализировать реализацию sync.Mutex
-

11.6.2. Практические задания (Staff Level)

№ 11.6.2.1

- **Задание:** Написать программу, анализирующую GC (включить логи)
- **Что проверяет:** Понимание GC

№ 11.6.2.2

- **Задание:** Проанализировать планировщик через трассировку (GODEBUG=schedtrace=1000)
- **Что проверяет:** Понимание планировщика

№ 11.6.2.3

- **Задание:** Написать lock-free структуру с использованием atomic
 - **Что проверяет:** Понимание атомарных операций
-

🔗 СВОДНЫЙ ЧЕК-ЛИСТ ДЛЯ ПОДГОТОВКИ К СОБЕСЕДОВАНИЮ

Junior Level (должен знать 100%)

Переменные, константы, функции

Управляющие конструкции (if, for, switch)

Структуры и методы

Указатели

Массивы и срезы

Map (хеш-таблицы)

Обработка ошибок

Пакеты и импорты

Интерфейсы (база)

Middle Level (должен знать 100%)

Goroutines и Channels

select, close

sync.WaitGroup, sync.Mutex, sync.RWMutex

context (отмена, таймауты)

HTTP-сервер (net/http)

JSON (encoding/json)

SQL (database/sql)

go mod (управление зависимостями)

go test (тестирование)

Senior Level (должен знать 100%)

Worker Pool, Fan-in/Fan-out, Pipeline

sync.Pool, sync.Once

atomic (атомарные операции)

errgroup

gRPC (Protocol Buffers)

Middleware в HTTP

GORM (ORM)

Redis, Kafka, RabbitMQ

Docker, K8s для Go

Prometheus, OpenTelemetry

Staff Level (должен знать 100%)

Планировщик Go (GMP, work stealing)

GC (триколорная маркировка, STW)

Внутренности: chan, interface, slice, map

sync.Mutex внутреннее устройство

pprof (профилирование)

Оптимизация производительности

Написание lock-free структур

🔨 ПРАКТИЧЕСКИЙ ПРОЕКТ ДЛЯ ЗАКРЕПЛЕНИЯ ДОМЕНА 11

Задание: Разработать микросервисную систему на Go для обработки заказов.

Шаги:

1. **Сервисы:** Order Service, Payment Service, Inventory Service (микросервисы)
2. **API:** REST API с chi (роутер) и gRPC
3. **БД:** PostgreSQL с GORM
4. **Очереди:** Kafka для асинхронных событий
5. **Кеш:** Redis для кеширования
6. **Контейнеризация:** Docker (multi-stage)
7. **Оркестрация:** Kubernetes (Deployment, Service, Ingress)
8. **Метрики:** Prometheus + Grafana
9. **Трейсы:** OpenTelemetry + Jaeger
10. **Тесты:** unit-тесты, интеграционные тесты
11. **CI/CD:** GitLab CI для сборки и деплоя
12. **Профилирование:** pprof для анализа производительности

Проверка:

- Система должна обрабатывать 5000 запросов/сек
- Время ответа < 100 мс
- Все тесты должны проходить
- Контейнер должен быть < 20 МБ